Computer Science Department

TECHNICAL REPORT

"An ICON Package for
Experimenting with Euclidean Domains"
by
Lars Warren Ericson

Technical Report #232
August, 1986

NEW YORK UNIVERSITY

Department of Computer Science
Courant Institute of Mathematical Sciences
251 MERCER STREET, NEW YORK, N.Y. 10012

An ICON package for experimenting with Euclidean domains

Lars Warren Ericson

Courant Institute of Mathematical Sciences
New York University
251 Mercer St.
New York, NY 10012

ARPA: ericson@nyu

May 16, 1986

Abstract

For the purpose of understanding the algebraic algorithms over the Euclidean domain presented in the book of Lipson [Lipso81a], a small package of routines (about 2000 lines of code) was written in ICON, a software prototyping language developed at the U. of Arizona [Grisw83a,Grisw83b]. This package allows the ICON user to write algorithms which apply to any object of a Euclidean domain, and supplies a paradigm for implementing new Euclidean domains. The package implements those Euclidean domains found in Lipson's book. It turns out that the most difficult part of such a package is the implementation of *div* and *mod* for an arbitrary domain. This led the author to exploit a feature of Icon Version 5.10 [Grisw85a] (function call by string image of name) in order to implement a "by-hand" version of the sort of call by inheritance seen in Smalltalk [Ingal78a] and Scratchpad II [Balza84a]. The package may be of use to algebraic algorithm prototypers in the ICON community, or as an adjunct to a course on computer algebra.

Contents

1. Introduction

1.1. Programming with Euclidean domains

John Lipson's book, *Elements of Algebra and Algebraic Computing*, presents a number of interesting symbolic algebraic algorithms, in a style which seems implementable. The only non-trivial implementation detail for the algorithms presented by Lipson is that they assume *div* and *mod* operations which are defined on every Euclidean domain (and, by implication, representations and definitions of these operations for every Euclidean domain). For example, consider his presentation of the FFT algorithm (p. 298):

```
procedure FFT(N,a(x),ω,A);
 if N = 1
    then
       { Basis. } A0 := a0
    else
    begin
       { Binary split. }
          n := N / 2;
          b(x) := Σ(i=0 to n-1) a2i x^i;
          c(x) := Σ(i=0 to n-1) a2i+1 x^i;
       { Recursive calls. }
          FFT(n, b(x), ω^2, B);
          FFT(n, c(x), ω^2, C);
       { Combine. }
          for k := 0 until n - 1 do
          begin
             Ak := Bk + ω^k⊗Ck;
             Ak+n := Bk - ω^k⊗Ck;
          end
    end
```

The purpose of the package of routines described in this paper is to allow an ICON user to implement an algorithm such as FFT, at about the same level of description as above. By comparison, see Section 3.3.2, which contains our ICON version of the same procedure.

In order to support a high level of description, it must be possible to describe the implementation of particular Euclidean domains, and to describe algorithms which apply generically to all Euclidean domain instances. We do this by deciding which functions are

expected of all Euclidean domain implementations (say, *div*, *mod*, + and -), and then implementing a "dispatch" version of each of these. The "dispatch" *div* function inspects the type of its argument (say, integer, polynomial, quotient domain element or modular domain element), and then calls the associated *div* function in the domain implementation (say div_integer, div_poly, div_Q or div_mod).

The ability to test the run-time environment is a feature of ICON. Given a string, say "X", and an integer corresponding to a number of formal parameters, say 3, *proc("X", 3)* will return a procedure (a first-class value in ICON, assignable to variables) if the identifier *X* is globally to a procedure which is defined to take 3 arguments. Otherwise *proc* fails. To test for the procedure $\otimes_Z$, we evaluate *proc("times" || "_Z", 2)*, and in general, for some string value X which corresponds to a procedure name, Y a domain name, and i a number of formal parameters, we evaluate *proc(X || "_" Y, i)*, where || is the ICON string concatenation operator. For example, here is the code for the "generic" division operation:

```
⊖ (a, b) ⇐  ⇑ proc("div_"||type(a), 2)(a, b) ■
```

Every implementation of a Euclidean domain must supply certain required procedures. (This notion of "must" corresponds to the idea of a "category" in Scratchpad II.) Optional procedures may be supplied by the domain implementation, but are synthesized if not supplied. The following table lists required, optional and synthesized procedures.

BASIC PROCEDURES FOR COMPUTING WITH DOMAINS			
Type	*Required*	*Optional*	*Synthesized*
Constant	**0**		
	1		
Operator	abs		
	$\oplus$		
	$-$		
	$\otimes$		
	$\oplus$		
		mod	
		rem	
		normalize	
			exp
Predicates	=		
		<	
	<**0**		
	unit		
	=**0**		
Commands		print	
			pr, prs

For a typical domain implementation, which serves as a model for other domain implementations, see for example Section 2.3.1, which describes our implementation of Quotient domains.

A typical application is our implementation of Lipson's algorithm (p. 264) for Newton Interpolation, seen in Section 3.3.3.

1.2. A summary of package facilities for Euclidean domains

The domains supported are as follows:

EUCLIDEAN DOMAIN CONSTRUCTIONS		
Primitive domains	*integer*	Machine word integers
	$base_B$	Arbitrary precision unsigned base B integers
	Z	Signed infinite precision integers
Domain constructors	Q	Quotient domain
	modulo	Modular domain
	poly	Polynomial domain
	tpower	Truncated power series domain

The following are the representations in ICON we have adopted for objects in the Euclidean domains we support:

Domain representation	
integer	integer
$base_B$	record base_b (base, digits)
Z	record Z (sign, mantissa)
Q	record Q (dividend, divisor)
modulo	record modulo (item, modulus)
poly	record poly (terms) record term (coef, power)
tpower	record tpower (poly, N)

We have implemented the following application algorithms, which may be applied to objects from any Euclidean domain (unless otherwise noted):

Application Algorithms	
GCD	greatest common divisor
EUCLID	extended GCD
MOD_RS	polynomial remainder sequence for GCD
PREM	integral domain remainder
E_PRS	polynomial remainder sequence for PREM
INVERSE	inverse of x mod y
NIA	Newton interpolation algorithm
CRA2, CRA	Chinese remainder algorithm for 2 or more linear congruences
FFT	Fast Fourier Transform
FFI	Fast Fourier Interpolation
NPSI	Newton power series inversion for truncated power series

The system as described is comprised of about 2000 lines of commented ICON code. Supposing that the code defined in the following sections is stored in a file, say `euclid`, then it may be executed in ICON by adding the statement `link euclid` to the application program, and then running the ICON translator. The author will gladly supply this code (as is) to any interested user. Mail to ARPA:ericson@nyu or UUCP:{floyd,ihnp4}!cmcl2!csd1!ericson for more information, or via U.S. Mail (with a 600 ft mag tape) to the address listed at the beginning of this report. (The offer last until the author gets sick of making tapes.)

1.3. Our typographical conventions for displaying ICON code

We have dressed up and compressed the syntax of ICON, to give the algorithms presented a more compact, functional appearance.

Icon variables (simple names for single items, and procedure names) may appear as subscripted quantities. This is purely formal, not actual, subscripting. Also, some operator symbols are defined which would not be legal identifiers in ICON (because the characters don't exist in ASCII). Rather than spelling them out, in this report we use the symbol we would have liked to use. The following are some examples of the original code and the fancier notation. Note that underscore ("_") is not a meta-character, but an ordinary character that may appear in identifiers in ICON:

Original ICON	Fancy Notation
one_base_B	1_{base_B}
delta_i_minus_1	δ_{i-1}
plus_poly	$\oplus_{poly}$

For procedure definitions, instead of the obvious

```
procedure F (a, b, c)
  code
end
```

we use the logical-looking

```
F (a, b, c) ⇐ code ■
```

For return x we use ⇑ x, and for return we use ⇑. Instead of fail we use ⊥. All other ICON reserved words are bold-faced.

1.4. Afterthoughts

This code is no longer under development, and seems to be primarily of educational value. In the future, code such as this will be supplanted by far more capable systems such as Scratchpad II, as they become widely available and inexpensive.

As an exercise, the author believes that the code addresses some of the essential software organization issues in computer algebra system design. If the code is to be applied in an educational setting, it would be well to stress to students several other important areas (these areas could form the basis of semester projects to extend the package):

- Algorithm design. For example, efficient polynomial greatest common divisor, which has seen many attempts to reduce its complexity. Zippel's notes [Zippe86a] contain a good discussion of this problem.

- Multivariate polynomials. This code does not supply any of the several multivariate polynomial representations.

- Explainability. Algorithmic (as opposed to "deductive") systems do not explain themselves. An ideal system would supply a proof its conclusion.

- Numeric-Symbolic Interface. The results of some computations, for example, polynomial root-finding[Yap86a], are best expressed as numeric approximation intervals, even though they are defining "symbolic" quantities. More work needs to be done to automate the relationship between approximate versus exact computations.

2. Euclidean domains: representation and basic arithmetic

2.1. Generic arithmetic for Euclidean domains

Lipson's book, p. 203, contains a significant proviso:

> We assume that our (Algol-like) language allows for the manipulation of values from an arbitrary Euclidean domain *D* with degree function *d*. In particular we assume that our language provides a *Division Algorithm* in the form of two operations "div" and "mod" which return, respectively, a preferred quotient and remainder in accordance with the Division Property of a Euclidean domain...

The purpose of this package is to partially implement this proviso. The package implements several primitive domains and *domain constructors*, which are classes of domains composed from other domains.

When a procedure like $\oplus$ or *mod* is applied to an object which is an instance of a Euclidean domain, the type of the object is determined by inspection. This is either the primitive type, in the case of an instance of a primitive domain, or the type of the "outermost" constructor, in the case of an instance of a composite domain. In the case of required and optional procedures, the run-time environment is then tested to determine whether the domain implementation supplies an operation of this type. If the name of the domain is D, and the procedure name is P, then the run-time environment is tested for a procedure named P_D. For example, $\oplus$ applied to a quotient will look up the procedure $\oplus_Q$. Required procedures must be defined by the domain implementation, otherwise the operation fails. Implementation-optional procedures will synthesize their values if a more domain-specific implementation does not exist.

Constants.

A consequence of the existence of a variety of Euclidean domain instances is that there are a variety of structural representations for 0 and 1. In a given computation, the 0 or 1 used must be of the type of the domain instance. Hence to obtain the correct 0, we evaluate a 0 function which, given an object of the domain instance, returns the 0 of that domain, and similarly for 1.

```
0 (a) ⇐  ↑proc("zero_"||type(a),1)(a) ■

1 (a) ⇐  ↑proc("one_"||type(a),1)(a) ■
```

Operators.

Euclidean domains: representation and basic arithmetic

The following procedures define the basic arithmetic operations for domains. As noted in Table 1, every domain must supply *Abs*, $\oplus$, $-$, $\otimes$ and $\oslash$. *mod*, *rem* and *normalize* are optional, and $\ominus$ and exp are synthesized.

```
Abs (a) ⇐  ↑ proc("Abs_"||type(a),1)(a) ■

⊕ (a, b) ⇐  ↑ proc("plus_"||type(a), 2)(a, b) ■

⊖ (a, b) ⇐ ↑ ⊕(a, −(b)) ■

− (x) ⇐  ↑ proc("minus_"||type(x), 2)(x) ■

⊗ (a, b) ⇐  ↑ proc("times_"||type(a), 2)(a, b) ■

⊘ (a, b) ⇐  ↑ proc("div_"||type(a), 2)(a, b) ■

mod (a, b) ⇐
  if (x := proc("mod_"||type(a), 2)(a, b)) then ↑ x
  if <(b, 0(b)) then ↑ mod(a, −(b))
  ↑ normalize(
        if <(a, 0(a))
        then ⊕(a, ⊗(b, ⊕(⊘(−(a), b), 1(a))))
        else ⊕(a, −(⊗(b, ⊘(a, b)))))
■
```

Example. The polynomials

$$a(x)=x^3-2$$

$$b(x)=2x^2-3$$

in the domain of quotients of machine-word integers are denoted within ICON by the record-constructor expressions and variable assignments

```
ax := poly([term(Q(-2,1), 0), term(Q(1,1), 3)])
bx := poly([term(Q(-3,1), 0), term(Q(2,1), 2)])
```

pr, a printing control structure, causes expressions to be printed out in a pleasing fashion. The ICON expression *pr*{*ax*, " *mod* ", *bx*, " = ", *mod(ax, bx)*} will print the following result:

```
(-2)q + 1q*X^3 mod (-3)q + 2q*X^2 = (-2)q + (3/2)q*X
```

Similarly, given $c(x)=(3/2)x-2$, represented as

```
cx := poly([term(Q(-2,1), 0), term(Q(3,2), 1)])
```

The result of evaluating *pr*{*bx*, " *mod* ", *cx*, " = ", *mod(bx, cx)*} is

```
(-3)q + 2q*X^2 mod (-2)q + (3/2)q*X = (5/9)q
```

```
rem (a, b) <=
  |^ (if (x := proc("rem_"||type(a), 2)(a, b)) then x
        else ⊖(a, ⊗(⊕(a, b), b)))
```

■

Example. The polynomials

$$a(x)=5-2x+x^2$$

$$b(x)=2$$

in the domain of quotients of machine-word integers are denoted with ICON by

```
ax := poly([term(Q(5, 1), 0), term(Q(-2, 1), 1), term(Q(1, 1), 2)])
bx := poly_of(Q(2, 1))
```

The result of evaluating *pr{ax, " rem ", bx, " = ", rem(ax, bx)}* is

```
5q + (-2)q*X + 1q*X^2 rem 2q = 0q
```

Similarly, given the equations over the integral domain of polynomials over machine integers

$$a(x)=8-9x+6x^2$$

$$b(x)=3$$

denoted by

```
ax := poly([term(8, 0), term(-9, 1), term(6, 2)])
bx := poly_of(3),
```

the result of evaluating *pr{ax, " rem ", bx, " = ", rem(ax, bx)}* is

```
8 + (-9)*X + 6*X^2 rem 3 = 2
```

normalize returns a preferred normal form of a value for a given domain. For example, for quotients, it would be the quotient such that the dividend and divisor have no common non-unit factors. For a modular domain, it would be the least positive element of the equivalence class of the value.

```
normalize (a) <=
  if (x := proc("normalize_"||type(a), 1)(a)) then |^ x
  |^ a
■
```

exp is the Russian Peasants algorithm for exponentiation. Our version is transliterated from R.B.K. Dewar's SETL implementation of arithmetic for the NYU Ada/Ed system

[Dewar81a,Kruch83a].

```
exp (x, p) ⇐
 if p = 1 then ⇑x
 else { result := 1(x)
        u := copy(x); v := p
        running := u
        while v ~= 0 do
        {    if v % 2 = 1 then result := ⊗(result, running)
             running := ⊗(running, running)
                 v := v / 2    }
        ⇑result    }
■
```

Predicates.

All of the predicates defined below except | are required to be defined by a domain instance implementation if they are to be used. However, this is not a minimal set: for example, *is_zero* could be defined in terms of =. | is really not a basic predicate, but since it may be defined in a general way, we include it here.

```
= (a, b) ⇐  ⇑proc("equal_"||type(a),2)(a, b) ■

< (a, b) ⇐ ⇑((proc("less_"||type(a),2)(a, b)) | <0(⊖(a, b))) ■

<0 (x) ⇐  ⇑proc("negative_"||type(x),1)(x) ■

unit (x) ⇐  ⇑proc("unit_"||type(x),1)(x) ■

=0 (x) ⇐  ⇑proc("is_zero_"||type(x),1)(x) ■
```

$a|c$ (a divides c) if c is a multiple of a, that is, if $rem(c,a)=0$.

```
| (a, c) ⇐ ⇑ =0(rem(c, a)) ■
```

Commands.

Every domain instance *D* implementation should define a preferred method of printing values in the domain, $print_D$. On top of this, we supply printing control structures *pr* and *prs*. *pr* takes a list of arguments enclosed in braces, and prints them, using the printing procedure appropriate for the type of each argument, followed by a carriage return. *prs* is the same, omitting the carriage return.

prs and *pr* are defined using the user-defined control operation features of ICON 5.10. [Grisw85a, Grisw83a] When *pr* or *prs* is called with a sequence of expressions in braces, the expressions are passed as unactivated co-expressions, which are then activated with the ICON @ operator.

```
prs (x) <= every y := !x do print(@y) ■

pr (x) <=
  (every y := !x do print(@y))
  write()
■

print (x) <=
  if type(x) == "list"
  then { writes("[")
  every y := !x[1:*x] do { print(y); write(", ") }
  print(x[*x]); writes("]")      }
  else if pp := proc("print_"||type(x), 1) then pp(x)
  else if type(x) == "string" then writes(x)
  else writes(imagex(x))
■
```

2.2. Primitive domains

The *primitive* domains are those which are not constructed from other domains, or which are best thought of as undecomposable. We have three such domains available:

- Arbitrary-precision arbitrary-base integers.
- Arbitrary-precision base 10 integers.
- Ordinary machine integers.

The latter are best unused: ICON does not notify the user of integer multiplication overflow, and overflow can occur very easily in the applications we deal with. For example, subresultant polynomial remainder sequences with cofficients in the 10000 range involve intermediate calculations in the 10000^2 range.

2.2.1. Abitrary base, infinite precision non-negative integer arithmetic

Base B Arithmetic Facilities	
Data structures	$base_B$; set_{base}
Constants	0_{base_B}, 1_{base_B}, k_{base_B}
Operators	$\oplus_{base_B}$, $\ominus_{base_B}$, $\otimes_{base_B}$, $\oslash_{base_B}$, $normalize_{base_B}$
Predicates	$<_{base_B}$, $=_{base_B}$
Commands	$print_{base_B}$

Data structures.

base is a number B such that B^2-1 is less than the maximum machine word integer. Then *digits* is a list of machine word integers less than *base* and greater than 0. Width is the printing width of digits of the base, in terms of decimal digits.

record $base_B$ (base, digits)

global Base, Width

set_{base} (b, w) ⇐

```
Base := b
Width := *(b||"") - 1
■
```

Constants.

0_{base_B} (x) ⇐ ⇑ $base_B$(x.base, [0]) ■

1_{base_B} (x) ⇐ ⇑ $base_B$(x.base, [1]) ■

k_{base_B} (x) ⇐ ⇑ $base_B$(Base, *digits_of*(abs(x), Base)) ■

digits_of (x, B) ⇐ if x < B then ⇑ [x] else ⇑ *digits_of*(x/B ,B) ||| [$mod_{integer}$(x,B)] ■

Operators.

The base B addition algorithm is that of Lipson, p. 199. For input it takes a, b, lists of integers $\leq B$, of length m returning $a+b$.

```
⊕base_B (a, b) ⇐
  B := a.base
  ⇑ base_B(B, ⊕digits(a.digits,b.digits,B))
■

⊕digits (ad, bd, B) ⇐
  m := *ad; n := *bd
  if m < n then { a := (list(n-m,0) ||| ad); b := bd }
  else if m > n then { a := ad; b := list(m-n,0) ||| bd }
  else { a := ad; b := bd }
  m := *a;
  c_digits := list(m+1, 0);
  gamma := 0
  every i := m to 1 by -1 do
  {     t := a[i] + b[i] + gamma
        c_digits[i+1] := mod_integer(t, B)
        gamma := t / B                    }
  c_digits[1] := gamma
```

```
⇑ normalize_digits(c_digits)
```

■

Example. The result of evaluating

```
x := base_B(8,[1]); y := base_B(8,[7,7,7])
pr{x, " + ", y, " = ", ⊕_base_B(x, y)}
```

is

```
1 #8# + 7 7 7 #8# = 1 0 0 0 #8#
```

The base B subtraction algorithm is Knuth Algorithm 4.3.1 S, transliterated from a SETL implementation of Robert Dewar. Assume $a \geq b$ are lists of integers $\leq B$. Returns $a-b$.

```
⊖_base_B (a, bb) ⇐
  b := copy(bb); B := a.base; m := #a.digits
  repeat
  { n := #b.digits
    if m < n then pr{"ERROR: base_B integer subtraction underflow"}
    else if m > n
    then b := base_B(B, list(m-n,0) ||| b.digits)
    else ⇑ base_B(b.base, ⊖_digits (a.digits, b.digits, b.base))}
```

■

```
⊖_digits (a, b, B) ⇐
  u := copy(a)
  v := list(#a-#b,0) ||| copy(b)
  k := 0
  every j := #u to 1 by -1 do
  {     u[j] := u[j] - v[j] + k
        if u[j] < 0 then { u[j] +:= B; k := -1 } else k := 0 }
```

```
⇑ normalize_digits(u)
■
```

Example. The result of evaluating

```
x := base_B(10, [1,0,0,5,6,3]); y := base_B(10, [5,3,3,5])
pr{x, " - ", y, " = ", ⊖_base_B(x,y)}
x := base_B(10,[2,1,2]); y := base_B(10, [9,9])
pr{x, " - ", y, " = ", ⊖_base_B(x, y)}
y := base_B(10, [1,9,9])
pr{x, " - ", y, " = ", ⊖_base_B(x, y)}
```

is

```
1 0 0 5 6 3 #10# - 5 3 3 5 #10# = 9 5 2 2 8 #10#
2 1 2 #10# - 9 9 #10# = 1 1 3 #10#
2 1 2 #10# - 1 9 9 #10# = 1 3 #10#
```

```
normalize_base_B (r) ⇐
  d := normalize_digits(r.digits)
  ⇑ base_B(r.base, d)
■
```

```
normalize_digits (d) ⇐
  while (#d > 1) & (d[1] = 0) do pop(d)
  ⇑ d
■
```

The base B multiplication algorithm is that of Lipson, p. 200. As input it takes a, b, lists of integers $\leq B$, of length m and n. It outputs $a \otimes b$.

```
⊗base_B (a, b) ⇐ ⇑ base_B(a.base, ⊗digits(a.digits, b.digits, a.base)) ■

⊗digits (a, b, B) ⇐
 m := *a
 n := *b
 c := list(m+n,0)
 every k := 0 to n-1 by 1 do
 {     gamma := 0
       every i := 0 to m-1 by 1 do
       {       t := a[m-i]*b[n-k] + c[m+n-k-i] + gamma
               if t < 0
               then pr{"ERROR: integer overflow in ⊗base_B, base = ", B}
               c[m+n-k-i] := mod_integer(t, B)
               gamma := t / B       }
       c[n-k] := gamma                }
 ⇑ normalize_digits(c)
■
```

Example. The result of evaluating

```
x := k_base_B(28107324); y := k_base_B(75625)
pr{x, " * ", y, " = ", ⊗base_B(x,y)}
x := k_base_B(28107324); y := k_base_B(75625)
pr{x, " * ", y, " = ", ⊗base_B(x,y)}
x := k_base_B(7478); y := k_base_B(4625)
pr{x, " * ", y, " = ", ⊗base_B(x,y)}
```

is

```
2 8 1 0 7 3 2 4 #10# * 7 5 6 2 5 #10# = 2 1 2 5 6 1 6 3 7 7 5 0 0 #10#
2 8 1 0 7 3 2 4 #10# * 7 5 6 2 5 #10# = 2 1 2 5 6 1 6 3 7 7 5 0 0 #10#
7 4 7 8 #10# * 4 6 2 5 #10# = 3 4 5 8 5 7 5 0 #10#
```

The following algorithm computes $\frac{a}{b}$ by long division. The design is that of Knuth Algorithm 4.3.1 D [Knuth73a], and the implementation is largely borrowed from a SETL implementation of Robert Dewar [NYU 84a]. Most of the following comments are lifted from the Dewar implementation.

This is by far the most difficult of the four basic operations. This is because the paper and pencil algorithm involves certain amounts of guess work which cannot be programmed directly. The approach (analyzed in detail by Knuth) is to reduce the guess work by computing a rather good guess at each digit of the result, and then correcting if the guess is wrong.

```
⊕base_B (a, b) ⇐ ⇑ normalize_base_B(base_B(a.base, ⊕digits (a.digits, b.digits, a.base))) ■

⊕digits (a, b, B) ⇐
```

If the divisor is 0, then fail.

```
if (#b = 1) & (b[1] = 0) then { pr{"ERROR: divide by 0 in base_B"}; ⊥ }
```

If a is shorter than b, return 0.

```
if #a < #b then ⇑ [0]
```

The case of a one digit divisor is treated specially. Not only is this more efficient, but the general algorithm assumes that the divisor contains at least two digits. Basically dividing by a single digit is straightforward. Since we can represent numbers up to $B*B-1$, we can do the steps of the division exactly without any need for guess work. The division is then done left to right.

```
if #b = 1 then
```

```
{   q := llst(*a, 0)
    rr := 0
    every j := 1 to *a do
    {       du := rr * B + a[j]
            q[j] := du  / b[1]
            rr := du % b[1]             }
    ⇑ normalize_digits(q)   }
```

Otherwise we must commence with the full long division algorithm.

```
u := copy(a)
v := copy(b)
n := *v
m := *u - n
q := llst(m+1, 0)
```

Knuth Step D1. [Normalize] The first step is to multiply both the divisor and dividend by a scale factor. Obviously such scaling does not affect the quotient. The purpose of this scaling is to ensure that the first digit of the divisor is at least B/2. This condition is required for the proper operation of the quotient estimation algorithm used in the division loop. Note that we added an extra digit at the front of the dividend above.

```
d := B / (v[1] + 1)
u := ⊗_digits (u, [d], B)
if *u = m+n then u := [0] ||| u
v := ⊗_digits (v, [d], B)
```

Knuth Step D2. [Initialize j] This is the major loop, corresponding to long division steps.

```
every j := 1 to m+1 do
{
```

Knuth Step D3. [Calculate q_hat] Guess the next quotient digit by doing a division based on the leading digits. This estimate is never low and at most 2 high.

```
if u[j] = v[1] then qe := B-1 else qe := ((u[j]*B)+u[j+1])/v[1]
```

The following loop refines this guess so that it is almost always correct and is at worst one too high (see Knuth [Knuth73a] for proofs).

```
while (v[2]*qe) > (((u[j]*B)+u[j+1]-(qe*v[1]))*B+u[j+2]) do qe-:= 1
```

Knuth Step D4. [Multiply and subtract] Now (for the moment accepting the estimate as correct), we subtract the appropriate multiple of the divisor. This is similar to the inner loop of the multiplication routine.

```
c := 0
every k := n to 1 by -1 do
{       du := u[j+k] - (qe * v[k]) + c
        u[j+k] := du % B
        c := du / B
        if u[j+k] < 0 then { u[j+k] +:= B; c -:= 1 }      }
u[j] +:= c
```

Knuth Step D5,D6. [Test remainder, Add back] If the estimate was one off, then $u[j]$ went negative when the final carry was added above. In this case, we add back the divisor once, and adjust the quotient digit.

```
q[j] := qe

if u[j] < 0 then
{       qe -:= 1
```

```
        c := 0
        every k := n to 1 by -1 do
        {       u[j+k] +:= v[k] + c
                if u[j+k] >:= B then { u[j+k] -:= B; c := 1 }
                else c := 0   }

        u[j] +:= c    }       }
```

⇑ $normalize_{digits}$(q)

■

Example. The result of evaluating

```
every xy := ![[10, 1], [4,2], [27, 9], [42,2], [90,1],
             [188175, 325], [188175, 579], [188175, 580],
             [188175, 578], [121903, 5335],
             [212, 99], [115668, 75625]]
    do { x := k_baseB(xy[1]); y := k_baseB(xy[2])
         pr{x, " / ", y, " = ", ⊕_baseB(x, y)} }
```

is

```
1 0 #10# / 1 #10# = 1 0 #10#
4 #10# / 2 #10# = 2 #10#
2 7 #10# / 9 #10# = 3 #10#
4 2 #10# / 2 #10# = 2 1 #10#
9 0 #10# / 1 #10# = 9 0 #10#
1 8 8 1 7 5 #10# / 3 2 5 #10# = 5 7 9 #10#
1 8 8 1 7 5 #10# / 5 7 9 #10# = 3 2 5 #10#
1 8 8 1 7 5 #10# / 5 8 0 #10# = 3 2 4 #10#
1 8 8 1 7 5 #10# / 5 7 8 #10# = 3 2 5 #10#
1 2 1 9 0 3 #10# / 5 3 3 5 #10# = 2 2 #10#
2 1 2 #10# / 9 9 #10# = 2 #10#
1 1 5 6 6 8 #10# / 7 5 6 2 5 #10# = 1 #10#
```

Commands.

We supply a print command.

```
print_base_B (b) ⇐
  local digits
  writes(b.digits[1], " ")
  every writes(right(!rest(b.digits), Width, "0"), " ")
  writes("#", b.base, "#")
■
```

Predicates.

We supply two predicates, $<_{base_B}$ and $=_{base_B}$:

```
<_base_B (a, b) ⇐ ↑↑ base_B(a.base, <_digits (a.digits, b.digits)) ■

<_digits (a, b) ⇐
  if *a < *b then ↑↑
  else if (*a > *b) then ⊥
  else if *a = 0 then ⊥
  else if (a[1] > b[1]) then ⊥
  else if (a[1] < b[1]) then ↑↑
  else ↑↑ <_digits(rest(a), rest(b))
■

=_base_B (a, b) ⇐ ↑↑ =_digits (a.digits, b.digits) ■

=_digits (a, b) ⇐
  if *a < *b then ⊥
  else if (*a > *b) then ⊥
  else if *a = 0 then ↑↑
  else if (a[1] ~= b[1]) then ⊥
  else ↑↑ =_digits(rest(a), rest(b))
■
```

```
rest (x) ⇐ if *x < 2 then ⇑ [] else ⇑ x[2:*x+1] ■
```

2.2.2. Arbitrary precision integer Euclidean domain *Z*

Integer Arithmetic Facilities	
Data structures	Z
Constants	0_Z, 1_Z, k_Z
Operators	$\oplus_Z$, $-_Z$, $\otimes_Z$, $\ominus_Z$, mod_Z, abs_Z, deg_Z, $normalize_Z$
Predicates	$=_Z$, $<_Z$, $unit_Z$, $>0_Z$, $<0_Z$, $=0_Z$
Commands	$print_Z$

Data structures.

sign is 1 or −1. *mantissa* is a base *Base* integer, where the *Base* is set by k_Z.

```
record Z (sign, mantissa)
```

Constants.

0_Z (a) ⇐ ⇑ $Z(1, 0_{base_B}$(a.mantissa)) ■

1_Z (a) ⇐ ⇑ $Z(1, 1_{base_B}$(a.mantissa)) ■

k_Z takes an ICON integer and transforms it into a *Z* constant.

```
kZ (x) ⇐
  initial set_base(10000)
  ⇑ Z(if x = 0 then 1 else x/abs(x),
        base_B(Base, digits_of(abs(x), Base)))
■
```

Operators.

```
⊕Z (a, b) ⇐
 if <0Z(a) & >0Z(b) then ⇑ ⊕Z (b, a)
 ⇑ normalizeZ(
        if =0Z(a) then b
   else if =0Z(b) then a
   else if (>0Z(a) & >0Z(b)) | (<0Z(a) & <0Z(b))
        then Z(a.sign, ⊕baseB(a.mantissa, b.mantissa, Base))
   else { # a > 0 and b < 0, so...
             if <baseB(a.mantissa, b.mantissa)
             then Z(-1, ⊖baseB(b.mantissa, a.mantissa, Base))
             else Z(1, ⊖baseB(a.mantissa, b.mantissa, Base))  })
```

■

Example. The result of evaluating

```
x := kZ(1); y := kZ(-999)
pr{x, " + ", y, " = ", ⊕Z(x, y)}
```

is

```
1z + (-999z) = (-998z)
```

```
−Z (x) ⇐ ⇑ normalizeZ(Z(-x.sign, x.mantissa)) ■
```

Example. The result of evaluating

```
x := kZ(212); y := kZ(-99)
pr{"-", x, " = ", −Z(x)}
pr{"-", y, " = ", −Z(y)}
```

is

```
-212z = (-212z)
-(-99z) = 99z
```

$\otimes_Z$ (a, b) $\Leftarrow$ $\Uparrow$ *normalize*$_Z$(Z(a.sign*b.sign, $\otimes_{base_B}$(a.mantissa, b.mantissa))) ■

Example. The result of evaluating

```
x := kZ(28107324); y := kZ(75625)
pr{x, " * ", y, " = ", ⊗Z(x,y)}
x := kZ(7478); y := kZ(-4625)
pr{x, " * ", y, " = ", ⊗Z(x,y)}
```

is

```
28107324z * 75625z = 2125616377500z
7478z * (-4625z) = (-34585750z)
```

$\oplus_Z$ (a, b) $\Leftarrow$ $\Uparrow$ *normalize*$_Z$(Z(a.sign/b.sign,$\oplus_{base_B}$(a.mantissa,b.mantissa,Base))) ■

Example. The result of evaluating

```
every xy := ![[10, 1], [121903, 5335], [115668, 75625]]
do { x := kZ(xy[1]); y := kZ(xy[2]); pr{x, " / ", y, " = ", ⊕Z(x, y)}}
```

is

```
10z / 1z = 10z
121903z / 5335z = 22z
115668z / 75625z = 1z
```

```
modZ (a, b) ⇐
 ⇑ (if <Z(b, 0Z(b)) then modZ(a, −Z(b))
        else if <Z(a, 0Z(a))
            then ⊕Z(a, −Z(⊗Z(b, ⊕Z(−Z(1Z(a)), ⊕Z(a, b)))))
        else ⊕Z(a, −Z(⊗Z(b, ⊕Z(a, b)))))
■
```

Example. The result of evaluating

```
x := kZ(121903); y := kZ(5335)
pr{x, " mod ", y, " = ", modZ(x, y)}
```

is

```
121903z mod 5335z = 4533z
```

```
absZ (x) ⇐ ⇑ Z(1, x.mantissa) ■
```

deg_Z (x) ⇐ ⇑x ■

$normalize_Z$ (x) ⇐ ⇑ (if $=0_Z$(x) then Z(1, x.mantissa) else x) ■

Predicates.

```
=Z (a, b) ⇐
  if (=0Z(a) & =0Z(b)) then ⇑
  else if a.sign ~= b.sign then ⊥
  else ⇑ =baseB(a.mantissa, b.mantissa)
■

<Z (a, b) ⇐
  if a.sign < b.sign then ⇑
  if a.sign > b.sign then ⊥
  if a.sign = 1 then ⇑ <baseB(a.mantissa, b.mantissa)
  if a.sign = -1 then ⇑ <baseB(b.mantissa, a.mantissa)
■
```

$unit_Z$ (x) ⇐ ⇑ (=Z(x, 1_Z(x)) | =Z(x, Z(-1,1_{base_B}(x.mantissa)))) ■

```
>0Z (x) ⇐ ⇑ ((x.sign = 1) & not =0Z(x)) ■

<0Z (x) ⇐ ⇑ ((x.sign = -1) & not =0Z(x)) ■
```

=0_Z (x) ⇐ ⇑ =$_{base_B}$(x.mantissa, 0_{base_B}(x.mantissa)) ■

Commands.

```
printZ (a) ⇐
  local digits
  if a.sign < 0 then writes("(-")
  digits := a.mantissa.digits
  every ch := !digits do writes(right(ch, Width, "0"))
  writes("z")
  if a.sign < 0 then writes(")")
■
```

2.2.3. Small integers Euclidean domain

We provide the following machine integer arithmetic facilities:

Machine Integer Arithmetic Facilities	
Constants	$0_{integer}$, $1_{integer}$
Operators	$\oplus$, $-_{integer}$, $\odot_{integer}$, $circleslash_{integer}$, $rem_{integer}$, $mod_{integer}$, $deg_{integer}$, $abs_{integer}$
Predicates	$=0_{integer}$, $<0_{integer}$, $=_{integer}$, $unit_{integer}$
Commands	$print_{integer}$

Constants.

We provide constants 0 and 1, as follows:

$0_{integer}$ (x) $\Leftarrow$ ⇑ 0 ■

$1_{integer}$ (x) $\Leftarrow$ ⇑ 1 ■

Operators.

$\oplus$ (a, b) $\Leftarrow$ ⇑ a + b ■

$-_{integer}$ (x) $\Leftarrow$ ⇑ -x ■

$\odot_{integer}$ (a, b) $\Leftarrow$ ⇑ a * b ■

$circleslash_{integer}$ (a, b) $\Leftarrow$ ⇑ a / b ■

$mod_{integer}$ (a, m) $\Leftarrow$

```
if m < 0 then m := -m
repeat if a < 0 then a := a+(abs(a/m)+1)*m else ⇑ a % m
⇑ r
```

■

rem is not *mod*, because *rem* may be negative, but *mod* is never negative.

$rem_{integer}$ (a, b) $\Leftarrow$ ⇑ a % b ■

$deg_{integer}$ (x) ⇐ ⇑ x ■

$abs_{integer}$ (x) ⇐ ⇑ abs(x) ■

Predicates.

$=0_{integer}$ (x) ⇐ ⇑ (x = 0) ■

$<0_{integer}$ (x) ⇐ ⇑ x < 0 ■

$=_{integer}$ (a, b) ⇐ ⇑ a = b ■

$unit_{integer}$ (x) ⇐ if ((x = 1) | (x = -1)) then ⇑ x ■

Commands.

$print_{integer}$ (x) ⇐ if x < 0 then writes("(", x, ")") else writes(x) ■

2.3. Domain constructors

EUCLID provides three classes of domain constructions: quotient domains Q_D, modular domains $D/(e)$, polynomials $D[x]$ and truncated power series $T(D[[x]])_n$.

2.3.1. Quotient Euclidean domain $\mathcal{Q}$

Quotient Domain Arithmetic Facilities	
Data structures	$\mathcal{Q}$
Constants	$0_{\mathcal{Q}}$, $1_{\mathcal{Q}}$, $k_i\mathcal{Q}_X$
Operators	$\oplus_{\mathcal{Q}}$, $-_{\mathcal{Q}}$, $\otimes_{\mathcal{Q}}$, $\oplus_{\mathcal{Q}}$, *mod*$_{\mathcal{Q}}$, *normalize*$_{\mathcal{Q}}$, *deg*$_{\mathcal{Q}}$
Predicates	$=_{\mathcal{Q}}$, *unit*$_{\mathcal{Q}}$
Commands	*print*$_{\mathcal{Q}}$

Data structures.

The domains $\mathcal{Q}$ are of the form $\mathcal{Q}=\{\frac{m}{n} \mid m, n\in D, n\neq 0\}$, for some Euclidean domain D. Elements of such a domain $\mathcal{Q}$ are quotients with a dividend and a divisor:

```
record Q (dividend, divisor)
```

Constants.

```
0Q (x) ⇐ ⇑ Q(0(x.dividend), 1(x.dividend)) ■

1Q (x) ⇐ ⇑ Q(1(x.dividend), 1(x.dividend)) ■

kiQX (i, j) ⇐ ⇑ term(Q(i, 1(i)), j) ■
```

Operators.

Let $a=\frac{p}{q}$, $b=\frac{p'}{q'}$. Then $a+b=\frac{x}{y}$ where $x=pq'\oplus p'q$, $y=qq'$:

```
⊕Q (a, b) ⇐
  local zz, top
```

```
top := ⊕(⊗(a.dividend, b.divisor), ⊗(b.dividend, a.divisor))
zz := 0(a.dividend)
⇑(if =(top, zz) then 𝒬(zz, 1(a.dividend))
      else normalize_𝒬(𝒬(top, ⊗(a.divisor, b.divisor))))
■

−_𝒬 (x) ⇐ ⇑ 𝒬(−(x.dividend), x.divisor) ■

⊗_𝒬 (a, b) ⇐ ⇑ normalize_𝒬(𝒬(⊗(a.dividend, b.dividend), ⊗(a.divisor, b.divisor))) ■

⊕_𝒬 (a, b) ⇐
 local zz
 zz := 0(b.dividend)
 if =(b.dividend, zz) then pr{"ERROR: divide by 0 in 𝒬"}
 else ⇑(if =(a.divisor, zz) then 0_𝒬(a)
          else normalize_𝒬(𝒬(⊗(a.dividend, b.divisor), ⊗(b.dividend, a.divisor))))
■
```

There are no remainders in quotient division.

```
mod_𝒬 (a, m) ⇐ ⇑ 0_𝒬(a) ■
```

normalize_𝒬(x) reduces the size of the dividend and divisor, and ensures that any negative sign is in the dividend. Let $g=GCD(x,y)$. Then $normalize_{\mathcal{Q}}(\frac{x}{y})=\frac{x \oplus g}{y \oplus g}$.

```
normalize_𝒬 (x) ⇐
 local g, top, bottom
 g := GCD(x.dividend, x.divisor)
 top := ⊕(x.dividend, g)
 bottom := ⊕(x.divisor, g)
 ⇑(if <0(bottom) then 𝒬(−(top), −(bottom))
      else 𝒬(top, bottom))
■
```

```
deg_Q (x) ⇐ ⇑x ■
```

Predicates.

$\frac{p}{q}=\frac{p'}{q'}$ if and only if $pq'=qp'$.

```
equal_Q (a, b) ⇐ ⇑ =(⊗(a.divisor,b.dividend), ⊗(b.divisor, a.dividend)) ■
```

Everything is a unit in Q.

```
unit_Q (x) ⇐ ⇑■
```

Commands.

```
print_Q (x) ⇐
  if =(x.divisor, 1(x.divisor))
  then prs{x.dividend, "q"}
  else prs{"(", x.dividend, "/", x.divisor, ")q"}
■
```

2.3.2. Modular Euclidean domain $D/(x)$

Modular Domain Arithmetic Facilities	
Data structures	modulo
Constants	$\mathbf{0}_{modulo}$, $\mathbf{1}_{modulo}$
Operators	$\oplus_{modulo}$, $-_{modulo}$, $\otimes_{modulo}$, $\oplus_{modulo}$, $normalize_{modulo}$, deg_{modulo}
Predicates	$=_{modulo}$, $unit_{modulo}$, $<\mathbf{0}_{modulo}$
Commands	$print_{modulo}$

Data structures.

An item from a modular domain, say Z_5, is specified by the item in the "base" domain, plus the modulus.

record modulo (item, modulus)

Constants.

$\mathbf{0}_{modulo}$ (a) $\Leftarrow$ $\Uparrow$ modulo (**0**(a.item), a.modulus) ■

$\mathbf{1}_{modulo}$ (a) $\Leftarrow$ $\Uparrow$ modulo (**1**(a.item), a.modulus) ■

Operators.

$\oplus_{modulo}$ (a, b) $\Leftarrow$ $\Uparrow$ $normalize_{modulo}$(modulo($\oplus$(a.item, b.item), a.modulus)) ■

$-_{modulo}$ (x) $\Leftarrow$ $\Uparrow$ $normalize_{modulo}$(modulo(−(x.item), x.modulus)) ■

$\otimes_{modulo}$ (a, b) $\Leftarrow$ $\Uparrow$ $normalize_{modulo}$(modulo($\otimes$(a.item, b.item), a.modulus)) ■

$\oslash_{modulo}$ (a, b) $\Leftarrow$ $\Uparrow$ $normalize_{modulo}$(modulo($\otimes$(a.item, INVERSE(b.item,b.modulus)), a.modulus

$normalize_{modulo}$ (x) $\Leftarrow$ $\Uparrow$ modulo (mod(x.item, x.modulus), x.modulus) ■

deg_{modulo} (x) $\Leftarrow$ $\Uparrow$ mod (x.item, x.modulus) ■

Predicates.

$=_{modulo}$ (a, b) $\Leftarrow$ $\Uparrow$ =(mod(a.item, a.modulus), mod(b.item, b.modulus)) ■

$unit_{modulo}$ (a) $\Leftarrow$ $\Uparrow$ =(mod(a.item, a.modulus), 1) ■

Nothing is negative in a modular domain.

$<\mathbf{0}_{modulo}$ (a) $\Leftarrow$ $\perp$ ■

Commands.

$print_{modulo}$ (x) $\Leftarrow$ prs{"(", x.item, " mod ", x.modulus,")"} ■

2.3.3. Polynomial Euclidean domain $D[x]$

Polynomial Domain Arithmetic Facilities	
Data structures	poly, term; poly_of, 0_{th}_coef, lead_coef
Constants	0_{poly}, 1_{poly}, kZ_Q, kZQ_X, kZ_X
Operators	$\oplus_{poly}$, $-_{poly}$, $\otimes_{poly}$, $\ominus_{poly}$, mod_{poly}, $eval_{poly}$, deg_{poly}, $-_{deg}$, $\oplus_{deg}$, $normalize_{poly}$
Predicates	$<_{degree}$, $=_{poly}$, $unit_{poly}$
Commands	$print_{poly}$

Data structures.

Polynomials $a(x) \in$ some domain $D[x]$ are finite sums of the form $a(x) = \sum_{i=0}^{m} a_i x^i$. They are represented as lists of terms, in increasing order of power, such that there is always at least one term, 0, if the polynomial is zero. Otherwise the least term may be of any degree.

```
record poly (terms)

poly_of (x) ⇐ ⇑poly([term(x, 0)]) ■
```

The coefficient of the constant term as an element of D, if there is a constant term, otherwise 0, may be obtained with:

```
0th_coef (fx) ⇐
  local a
  a := fx.terms[1]
  ⇑ (if a.power = 0 then a.coef else 0(a.coef))
■
```

The coefficient of the term with the highest degree may be obtained with:

```
lead_coef (ax) ⇐ ⇑ (ax.terms[*ax.terms]).coef ■
```

A term, say ax^n, is represented as $coef*X^{power}$. It is assumed that coefficient and indeterminate range over the same base domain, and that the power ranges over $\mathcal{N}$.

```
record term (coef, power)
```

Constants.

The zero of the base domain of a coefficient of the polynomial is obtained via:

```
0poly (p) ⇐
  z := 0(p.terms[1].coef)
  ⇑ poly([term(z, 0)])
■
```

Example. The result of evaluating

```
pr{"Q:     0 = ", 0poly(poly([term(Q(-2,1), 0)]))}
pr{"QZ:    0 = ", 0poly(poly([kZQX(-2, 0)]))}
```

is

```
Q:    0 = 0q
QZ:   0 = 0zq
```

The one of the base domain of a coefficient of the polynomial may be obtained with:

```
1poly (p) ⇐
```

```
  z := 1(p.terms[1].coef)
  ↑↑ poly([term(z, 0)])
■
```

An arbitrary-precision rational whole number is obtaind with:

```
kZQ (e) ⇐
  top := kZ(e)
  ↑↑ Q(top, 1Z(top))
■
```

An arbitrary-precision rational whole number-coefficient indeterminate ex^y is obtained with:

```
kZQX (e, y) ⇐ ↑↑ term(kZQ(e), y) ■
```

An arbitrary-precision integer-coefficient indeterminate ex^y is obtained with:

```
kZX (e, y) ⇐ ↑↑ term(kZ(e), y) ■
```

Operators.

```
⊕poly (a, b) ⇐
  local Terms, T, z
  Terms := ⊕terms(a.terms, b.terms)
  T := []; z := 0(a.terms[1].coef)
  every t := !Terms do if not =(t.coef, z) then T |||:= [t]
  ↑↑ (if *T > 0 then poly(T) else 0(a))
■

⊕terms (a, b) ⇐
```

```
local c_coef, at, ap, ac, bt, bp, bc
⇑(
   if *a = 0 then b
else if *b = 0 then a
else {      at := a[1]; ap := at.power; ac := at.coef
            bt := b[1]; bp := bt.power; bc := bt.coef
            if less(ap, bp)
            then { if = (ac, 0(ac))
                   then ⊕terms(rest(a), b)
                   else [at] ||| ⊕terms(rest(a), b) }
       else if =(ap, bp)
            then { c_coef := ⊕(ac, bc)
                   if = (c_coef, 0(c_coef))
                   then ⊕terms(rest(a), rest(b))
                   else [term(c_coef, ap)] |||
                        ⊕terms(rest(a), rest(b)) }
       else ⊕terms (b, a)   })
```

■

Example. The result of evaluating

```
ax := poly([term(Q(-2,1), 0), term(Q(1,1), 3)])
bx := poly([term(Q(-3,1), 0), term(Q(2,1), 3)])"-
fx := poly([kZQX(-2, 0), kZQX(1,3)])
gx := poly([kZQX(-3, 0), kZQX(2,3)])
pr{"Q:    (", ax, ") + (", bx, ") = ", ⊕poly(ax, bx)}
pr{"QZ:   (", fx, ") + (", gx, ") = ", ⊕poly(fx, gx)}
```

is

```
Q:    ((-2)q + 1q*X^3) + ((-3)q + 2q*X^3) = (-5)q + 3q*X^3
QZ:   ((-2z)q + 1zq*X^3) + ((-3z)q + 2zq*X^3) = (-5z)q + 3zq*X^3
```

```
−poly (x) ⇐
 local c
 c := []
 every t := !x.terms do c ||:= [−term(t)]
 ⇑ poly (c)
■

−term (t) ⇐ ⇑ term(−(t.coef), t.power) ■
```

Example. The result of evaluating

```
ax := poly([term(Q(-2,1), 0), term(Q(1,1), 3)])
fx := poly([kZQx(-2, 0), kZQx(1,3)])
pr{"Q:      - (", ax, ") = ", −(ax)}
pr{"QZ:     - (", fx, ") = ", −(fx)}
```

is

```
Q:     - ((-2)q + 1q*X^3) = 2q + (-1)q*X^3
QZ:    - ((-2z)q + 1zq*X^3) = 2zq + (-1z)q*X^3
```

```
⊗poly (a, b) ⇐ ⇑ ⊗poly terms(a, b.terms) ■

⊗poly terms (a, b_terms) ⇐
 ⇑ (if *b_terms = 0 then 0(a)
       else ⊕poly(⊗poly term (a, b_terms[1]),
                  ⊗poly terms (a, rest(b_terms))))
■

⊗poly term (a, b_term) ⇐
 ⇑ (if *a.terms < 2
```

```
        then poly([⊗term term(a.terms[1], b_term)])
        else ⊕poly (poly([⊗term term(a.terms[1], b_term)]),
                    ⊗poly term (poly(rest(a.terms)), b_term)))
■

⊗term term (a_term, b_term) ⇐ ⇑term (⊗(a_term.coef, b_term.coef), a_term.power + b_term.po
```

Example. The result of evaluating

```
ax := poly([term(Q(-2,1), 0), term(Q(1,1), 3)])
bx := poly([term(Q(-3,1), 0), term(Q(2,1), 3)])
fx := poly([kZQx(-2, 0), kZQx(1,3)])
gx := poly([kZQx(-3, 0), kZQx(2,3)])
pr{"Q:      (", ax, ") * (", bx, ") = ", ⊗(ax, bx)}
pr{"QZ:     (", fx, ") * (", gx, ") = ", ⊗(fx, gx)}
```

is

```
Q:   ((-2)q + 1q*X^3) * ((-3)q + 2q*X^3) = 6q + (-7)q*X^3 + 2q*X^6
QZ:  ((-2z)q + 1zq*X^3) * ((-3z)q + 2zq*X^3) = 6zq + (-7z)q*X^3 + 2zq*X^6
```

```
⊕poly (a, b) ⇐
 local n, m, r, q, quotient
 n := degpoly(b)
 r := copy(a)
 quotient := 0poly(r)
 repeat {     m := degpoly(r)
              if <degree(m, n)
              then ⇑quotient
              else { q := poly([term(⊕(leadcoef(r),leadcoef(b)),m-n)])
```

```
if m = 0
then ⇑ ⊕poly(quotient, q)
else { subtrand := −poly(⊗poly(q, b))
       r := ⊕poly(r, subtrand)
       quotient := ⊕poly(quotient, q)      }}}
```

■

Example. The result of evaluating

```
ax := poly_of(1); bx := poly_of(3)
pr{"integers: ", ax, "/", bx, " = ", ⊕poly(ax, bx)}
ax := poly([term(Q(5,9), 0)])
bx := poly([term(Q(-2,1), 0), term(Q(3,2), 1)])
fx := poly([term(Q(kZ(5),kZ(9)), 0)])
gx := poly([term(Q(kZ(-2),kZ(1)), 0), term(Q(kZ(3),kZ(2)), 1)])
pr{"Q:     (", ax, ") / (", bx, ") = ", ⊕poly(ax, bx)}
pr{"QZ:    (", gx, ") / (", fx, ") = ", ⊕poly(gx, fx)}
ax := poly([term(Q(kZ(166), kZ(243)), 0), term(Q(kZ(-275),kZ(243)),1)])
bx := poly([term(Q(kZ(115668), kZ(75625)), 0)])
pr{"QZ[x]: (", ax, "/ ", bx, ") = 0", ⊕(ax, bx)}
```

is

```
integers: 1/3 = 0
Q:     ((5/9)q) / ((-2)q + (3/2)q*X) = 0q
QZ:    ((-2z)q + (3z/2z)q*X) / ((5z/9z)q) = ((-18z)/5z)q + (27z/10z)q*X
QZ[x]: ((166z/243z)q + ((-275z)/243z)q*X / (115668z/75625z)q) =
       (6276875z/14053662z)q + ((-20796875z)/28107324z)q*X
```

```
mod_poly (a, b) ⇐ ↑ ⊖(a, ⊗(b, ⊕(a, b))) ■
```

Evaluate $f(x)$ at a, that is evaluate $f(a)$:

```
eval_poly (fx, a) ⇐
  local r
  r := 0(a)
  every x := !fx.terms do r := ⊕(r, eval_term(x, a))
  ↑ r
■
```

Evaluate cx^p at $x=a$:

```
eval_term (t, a) ⇐ ↑ ⊗(t.coef, exp(a, t.power)) ■
```

Degrees of polynomials are values which may be integers, or the string "$-\infty$". Accordingly, special subtraction and addition procedures are required.

```
deg_poly (x) ⇐
  if =poly(x, 0poly(x)) then ↑ "- infinity"
  else ↑ x.terms[*x.terms].power
■

-deg (a, b) ⇐
  ↑ (if type(a) == "string" then b
       else if type(b) == "string" then a
       else a - b)
■

⊕deg (a, b) ⇐
  ↑ (if type(a) == "string" then b
       else if type(b) == "string" then a
```

```
        else a + b)
■
```

A normal-form polynomial is one whose terms are in normal form (and in ascending order of power).

```
normalize_poly (x) ⇐
  ts := []
  every t := !x.terms do ts |||:= [term(normalize(t.coef), t.power)]
  ⇑ poly(ts)
■
```

Predicates.

```
<_degree (a, b) ⇐
  if type(a) == "string"
  then ⇑ not (type(b) == "string")
  else ⇑ a < b
■

=_poly (a, b) ⇐ ⇑ =_terms (a.terms, b.terms) ■

=_terms (a, b) ⇐
  if *a ~= *b then ⊥
  if *a = 0 then ⇑
  if =_term(a[1], b[1]) then ⇑ =_terms(rest(a), rest(b))
■

=_term (a, b) ⇐ ⇑ (=(a.coef, b.coef) & =(a.power, b.power)) ■

unit_poly (x) ⇐ ⇑ ((*x.terms = 1) & (x.terms[1].power = 0) & unit(x.terms[1].coef)) ■
```

Commands.

```
print_poly (x) ⇐
  print_term(x.terms[1])
  every t := !rest(x.terms) do { writes(" + "); print_term (t) }
■

print_term (x) ⇐
  print(x.coef)
  if x.power = 1 then writes("*X")
  else if x.power > 1 then prs{"*X^", x.power}
■
```

2.3.4. Truncated Power Series domain $T(F[[x]])_n$.

Truncated Power Series Domain Arithmetic Facilities	
Data structures	tpower
Constants	$\mathbf{0}_{tpower}$, $\mathbf{1}_{tpower}$
Operators	$\oplus_{tpower}$, $-_{tpower}$, $\otimes_{tpower}$, $\oplus_{tpower}$, $normalize_{tpower}$
Predicates	$=_{tpower}$, $unit_{tpower}$
Commands	$print_{tpower}$

Data structures.

```
record tpower (Poly, N)
```

Constants.

The zero of the base domain of a coefficient of the polynomial:

```
0_tpower (x) ⇐ ↑(tpower(0_poly(x.Poly), x.N)) ■
```

The one of the base domain of a coefficient of the polynomial:

```
1_tpower (x) ⇐ ↑ tpower(1_poly(x.Poly), x.N) ■
```

Operators.

$\oplus_{tpower}$ (a, b) $\Leftarrow$ $\Uparrow$ tpower($\oplus_{poly}$(a.Poly, b.Poly), a.N) ■

$-_{tpower}$ (x) $\Leftarrow$ $\Uparrow$ tpower($-_{poly}$(x.Poly), x.N) ■

truncate (p, n) $\Leftarrow$ $\Uparrow$ poly(p.terms[1:n+1]) ■

$\otimes_{tpower}$ (a, b) $\Leftarrow$ $\Uparrow$ tpower(truncate($\otimes_{poly}$(a.Poly, b.Poly), a.N), a.N) ■

$\ominus_{tpower}$ (a, b) $\Leftarrow$ $\Uparrow$ tpower(truncate($\ominus_{poly}$(a.Poly, b.Poly), a.N), a.N) ■

$normalize_{tpower}$ (x) $\Leftarrow$ $\Uparrow$ tpower($normalize_{poly}$(x.Poly), x.N) ■

Predicates.

$=_{tpower}$ (a, b) $\Leftarrow$ $\Uparrow$ (a.N = b.N) & $=_{poly}$(a.Poly, b.Poly) ■

$unit_{tpower}$ (x) $\Leftarrow$ $\Uparrow$ $unit_{poly}$(x.Poly) ■

Commands.

$print_{tpower}$ (x) $\Leftarrow$ $print_{poly}$(x.Poly) ■

3. Algorithms for various problems over Euclidean domains

We provide algorithms for number of application areas:

- GCD, linear congruences and Diophantine equations.
- Polynomial remainder sequences.
- Power series and polynomial inversion and interpolation.

In addition, we provide a simple timer facility.

3.1. GCD, Linear Congruences and Diophantine Equations

We provide algorithms for the following applications:

- Euclid's algorithm for greatest common divisor, in simple and extended versions.
- Inverse of *a mod m*.
- The Chinese Remainder for 1, 2 or *N* congruences.
- The solutions to the Diophantine equation ax + by = c.

3.1.1. Greatest Common Divisor

We have two versions of Euclid's Algorithm over a Euclidean domain D, from Lipson, p. 226 and p. 209.

GCD(*a*,*b*,*D*)
Input: $a,b \in D$, not both zero.
Output: a gcd of *a*,*b*.

```
GCD (a, b) ⇐ ⇑ (If =(b, 0(b)) then normalize(a) else GCD(b, mod(a, b))) ■
```

The following is a table of expressions and their *gcd*'s, as computed via *GCD*:

Greatest Common Divisors

Domain	A	B	GCD
Z	121903z	5335z	1z
Z	-18z	5z	1z
Z	228z	612z	12z
Q[x]	(-2)q + 1q*X^3	(-3)q + 2q*X^2	(5/9)q
Z5	((-2) mod 5)	((-3) mod 5)	(2 mod 5)
Z5[x]	((-2) mod 5)+(1 mod 5)*X^3	(-3) mod 5)+(2 mod 5)*X^2	(3mod5)+(4mod5)*X
QZ[x]	(166z/243z)q + ((-275z)/243z)q*X	(115668z/75625z)q	(115668z/75625z)q
QZ[x]	(-2z)q + 1zq*X^3	(-3z)q + 2zq*X^2	(5z/9z)q

EUCLID(a,b)

Input: $a,b \in D$, not both zero.

Output: g,s,t such that g is a gcd of a, b and $g=sa+tb$.

```
EUCLID (A, B) ⇐
 local q, a, s, t
 a := [copy(A), copy(B)]
 s := [1(A), 0(A)]
 t := [0(A), 1(A)]
 while not(=(a[2], 0(A))) do
 {      q := ⊕(a[1], a[2])
        a := [a[2], ⊖(a[1], ⊗(a[2], q))]
        s := [s[2], ⊖(s[1], ⊗(s[2], q))]
        t := [t[2], ⊖(t[1], ⊗(t[2], q))] }
 ⇑ [normalize(a[1]), normalize(s[1]), normalize(t[1])]
■
```

The following is a table of expressions and their extended *gcd*'s, as computed via *EUCLID*:

Extended Greatest Common Divisors	
A, B	**GCD, s, t**
2, 4	2,1, 0
228,612	12, (-8), 3
59, 24	1, 11, (-27)
-2q+1q*X^3, -3q+2q*X^2)	(5/9)q, (-16/9)q+(-4/3)q*X, 1q+(8/9)q*X+(2/3)q*X^2
(-2mod5)+(1mod5)*X^3, (-3mod5)+(2mod5)*X^2)	(3mod5)+(4mod5)*X, (1mod 5), (2mod5)*X

3.1.2. Modular Inverse

Our modular inverse algorithms is that of Lipson, p. 214.

INVERSE(a,m): Computation of $a^{-1} mod m$

Input: $a,m \in D$, where D is a Euclidean domain. Output: If $(m,a)=1$, then $a^{-1} mod m$; otherwise error.

```
INVERSE (a, m) ⇐
local gst
gst := EUCLID(m, a)
if unit (gst[1]) then ⇑ mod (⊖ (gst[3], gst[1]), m)
else pr{"ERROR: ", a, "^-1 ", " mod ", m, " does not exist"}
■
```

A table of modular inverses as computed by *INVERSE* is as follows:

x	modulus	x^{-1}
30	197	46
16	21	4
18	21	ERROR
24	59	32
(1mod2)+(1mod2)*X^2	(1mod2)+(1mod2)*X^2+(1mod2)*X^5)	(1mod2)+(1mod2)*X+ (1mod2)*X^2+(1mod2)*X^4
(-3)q + 2q*X^2	(-2)q + 1q*X^3)	(9/5)q + (8/5)q*X + (6/5)q*X^2

3.1.3. Chinese Remainders and Single-Variable Linear Congruential Systems

We provide three algorithms, *CRA*1 for solving equations of the form $a\,x \equiv b \bmod m$, and *CRA*2 and *CRA* for solving systems of equations of 2 or more congruences of the form $x \equiv a \bmod m$.

CRA1 (a, b, m): Solution of a single linear congruence relation.

Input: a,b,m such that $a\,x \equiv b \bmod m$. Output: a particular solution x_1.

Niven and Zuckerman [Niven80a], in their section 2.3 note that, given a congruence $ax \equiv b \bmod m$, we can reduce it to $m\,y \equiv -b \bmod a$. If y_0 is a solution of the reduced congruence, then $x_0 = \frac{(my_0+b)}{a}$ is a solution for the original congruence.

They apply the reduction until the congruence is solvable "by inspection". This we do not do. They also have some tricks for size reduction (on p. 43) we will not apply (due to laziness). Our "by inspection" termination condition will be to perform the reduction until $a \bmod m = 1$ or $b = 0$. Then we return $b \bmod a$, in a recursive setting which builds up the original x_0.

```
CRA1 (aa, bb, m) ⇐
 local a, b, g
 g := GCD(aa, m)
 if not |(g, bb) then pr{"ERROR: no solution to linear congruence"}
 else { a := mod(aa, m); b := mod(bb, m)
        if =(a, 1(a)) then ⇑ b
        else if =(b, 0(b)) then ⇑ 0(b)
        else if =(a, b) then ⇑ 1(b)
        else ⇑ ⊕(⊕(⊗(m, CRA1 (m, −(b), a)), b), a) }
■
```

Example. The following results were obtained from executing *CRA*1 (the examples are from Niven and Zuckerman [Niven80a], Sect. 2.3:

- *CRA*(7,1432,5317): x such that $7x \equiv 1432 \bmod 5317$ is 4762.
- *CRA*(863,880,2151): x such that $863x \equiv 880 \bmod 2151$ is 173.
- *CRA*(589,509,817): There is no x such that $589x \equiv 509 \bmod 817$.

---

*CRA*2 and *CRA* are from Lipson, p. 254 and p. 257.

CRA2 (r, m, s, n): Two-congruence Chinese Remainder Algorithm for Z

Input: $r,m,s,n \in Z$, where n,m are relatively prime. Output: $U \in Z$ such that $U \equiv r \bmod m$, $U \equiv s \bmod n$.

```
CRA2 (r, m, s, n) ⇐
 local c, σ, U
 c := INVERSE(m, n)
 σ := mod(⊗(⊖(s, r), c), n)
 U := ⊕(r, ⊗(σ, m))
 ⇑ U
■
```

Example. The x such that $x \equiv 6 \bmod 7$ and $x \equiv 3 \bmod 9$ is 48, as obtained by evaluating *CRA*2(6,7,3,9).

CRA (rm_list): N-congruence Chinese Remainder Algorithm for Z

Input: $[[r_k, m_k]] \in Z$, where the m_k are relatively prime. Output: $U \in Z$ such that $U \equiv r_i \bmod m_i$.

```
CRA (rm_list) ⇐
 local rms, rm, M, U, c, σ
 rms := copy(rm_list)
 rm := pop(rms); r := rm[1]; m := rm[2]
 M := 1(m)
 U := mod(r, m)
 every k := 1 to *rms do
 {      M := ⊗(M, m)
        rm := pop(rms); r := rm[1]; m := rm[2]
        c := INVERSE(M, m)
```

```
    σ := mod(⊗(⊖(r, mod(U, m)), c), m)
    U := ⊕(U, ⊗(σ, M)) }
⇑U
■
```

Example. The problem is to find u(x) in Z[x] such that
$u(x) mod 3 = x$,
$u(x) mod 7 = 1$,
$u(x) mod 4 = 2x+3$, and
$u(x) mod 5 = 3x+3$

Let $u(x) = ax+b$. Then

a mod 3 = 1	b mod 3 = 0
a mod 7 = 0	b mod 7 = 1
a mod 4 = 2	b mod 4 = 3
a mod 5 = 3	b mod 5 = 3.

We can solve for a and b individually using the n-congruence CRA algorithm, and we are done. Executing the following code:

```
a_congruences := [[1, 3], [0, 7], [2, 4], [3, 5]]
b_congruences := [[0, 3], [1, 7], [3, 4], [3, 5]]
a := CRA(a_congruences)
b := CRA(b_congruences)
ux := poly([term(b, 0), term(a, 1)])
pr{"u(x) = ", ux}
```

we discover (final term due to Yap) that $u(x) = 183+238*X+3\cdot 7\cdot 4\cdot 5\sum_{i=0}^{\infty} t_i x^i$.

Example. Another example, from Lipson, p. 258, is to compute u such that
$u \equiv 1 (mod 3)$
$u \equiv 3 (mod 5)$
$u \equiv 0 (mod 7)$
$u \equiv 10 (mod 11)$
Executing the following code

```
pr{CRA([[1, 3], [3, 5], [0, 7], [10, 11]])}
```

yields a value of 868 for U.

3.1.4. Linear Diophantine Equations in Two Variables

According to Niven, sect. 5.2, $ax+by=c$ is solvable iff $g|c$ where $g=gcd(a,b)$. If $g|c$ then all solutions are of the form

$$x=x_1+(\frac{b}{g})t, y=y_1-(\frac{a}{g})t$$

where t is an arbitrary integer and $x=x_1$, $y=y_1$ is any particular solution of the equation. Particular solutions are obtained by solving one of the linear congruences

$$ax \equiv c \bmod |b|, by \equiv c \bmod |a|$$

for x_1 or y_1, then substituting y_1 or x_1 into $ax+by=c$ to obtain a particular y_1 or x_1. For computational convenience, if $|b| \leq |a|$, we solve the first congruence, otherwise we solve the second.

DIOPHANTINE (a, b, c): solves linear Diophantine equations in 2 variables.

Input: a,b,c such that $ax+by=c$. Output: g,x_1,y_1, described above.

```
DIOPHANTINE (a, b, c) ⇐
local gst, g, x1, y1
gst := EUCLID(a, b)
g := gst[1]; t := gst[3]
if not |(g, c) then pr{"ERROR: Diophantine solution nonexistent"}
else { if <(abs(b), abs(a))
          then { x1 := CRA1(a, c, abs(b))
                 y1 := ⊕(⊖(c, ⊗(a, x1)), b)  }
          else { y1 := CRA1(b, c, abs(a))
                 x1 := ⊕(⊖(c, ⊗(b, y1)), a) }
          ⇑[g, x1, y1]  }
```

■

Example. By evaluating *DIOPHANTINE*(84,54,−24), we find that all integer solutions (x,y) of the equation $84x+54y=(-24)$ are of the form $x=1+9t$, $y=(-2)-14t$.

Example. By evaluating *DIOPHANTINE*(999,−49,5000), we find that all integer solutions (x,y) of the equation $999x+(-49)y=5000$ are of the form $x=13+49t$, $y=163-(-999)t$.

Example. By evaluating *DIOPHANTINE*(247,589,817), we find that all integer solutions (x,y) of the equation $247x+589y=817$ are of the form $x=(-11)+31t$, $4y=6-13t$.

3.2 Polynomial remainder sequences

Polynomial remainder sequences are studied as a method of finding variants of the greatest common divisor for elements of integral domains. Variation in the definition is required because integral domains do not support long division. It is also desirable to compute values which share properties of the greatest common divisor (which might then be reclaimed by homomorphic image methods; see Lipson, ch. 8), such that the computation does not suffer the large coefficient growth of Euclid's algorithm on even moderate-sized polynomials. Yap [Yap85a] discusses the issue, presenting an example of Knuth exhibiting the coefficient growth problem. Polynomial remainder sequences are discussed in greater depth in the paper by Loos [Loosa].

We have implemented three variants of polynomial remainder sequence:

- *mod*-based PRS.
- *prem*, a pseudo-remainder for division over integral domains, and a *prem*-based PRS, as defined in Yap [Yap85a].
- Subresultant PRS, as defined in Yap [Yap85a] and based on an algorithm of Collins, as presented by Brown.

3.2.1 MOD-based PRS

The simplest polynomial remainder sequence is simply that of Euclid's algorithm. That is, we define *MOD_RS*(a,b) to be the PRS of *mod*(a,b).

```
MOD_RS (a, b) <= |^ [a] ||| (if =(b, 0(b)) then [b] else MOD_RS(b, mod(a, b))) |
```

Example. In $Q_Z[x]$, the remainder sequence of

$a(x)=x^5+2x^4+3x^2-x+2$

$b(x)=3x^3-x+2$

as encoded in ICON by

```
settime()
ax := poly([kZQx(2, 0), kZQx(-1, 1), kZQx(3, 2), kZQx(2,4), kZQx(1,5)])
bx := poly([kZQx(2, 0), kZQx(-1, 1), kZQx(3, 3)])
pr{"QZ[x]: MOD_RS(", ax, ",0", bx, ") = 0, MOD_RS(ax, bx)}
```

```
showtime()
```

is

```
QZ[x]: MOD_RS(2zq + (-1z)q*X + 3zq*X^2 + 2zq*X^4 + 1zq*X^5,
       2zq + (-1z)q*X + 3zq*X^3) =
[2zq + (-1z)q*X + 3zq*X^2 + 2zq*X^4 + 1zq*X^5,
2zq + (-1z)q*X + 3zq*X^3,
(16z/9z)q + ((-20z)/9z)q*X + 3zq*X^2,
(166z/243z)q + ((-275z)/243z)q*X,
(115668z/75625z)q,
0zq]
[221033 msecs]
```

■

3.2.2 Pseudo-remainder for division over integral domains

PREM(px, qx): Pseudo-remainder of $\frac{px}{qx}$, $px, qx \in I[x]$

where $I[x]$ is an integral domain.

Method:

1. Let $d = deg(p) - deg(q)$ 2. Let b = lead coefficient of $q(x)$ 3. Return $rem(b^{d+1}px, qx)$

```
PREM (px, qx) ⇐
 local d, b
 d := −deg (degpoly(px), degpoly(qx))
 b := poly_of(leadcoef (qx))
 ⇑ rem(⊗poly(exp (b, d+1), px), qx)
■
```

Example. The following table lists values and their pseudo-remainders.

Domain	$p(x)$	$q(x)$	$prem(p, q)$
QZ[x]	2042542724z+17851334z*X	58512592798467382524600000000000z	-58512592798467382524600000000000z
QZ[x]	21z+(-9z)*X+(-4z)*X^2+5z*X^4+3z*X^6	(-9z)+3z*X^2+(-15z)*X^4)	(-59535z)+30375z*X+15795z*X^2
QZ[x]	2zq+(-1z)q*X+3zq*X^2+2zq*X^4+1zq*X^6	2zq+(-1z)q*X+3zq*X^3	198zq+(-225z)q*X+306zq*X^2
QZ[x]	198zq+(-225z)q*X+306zq*X^3	18zq+369zq*X	10497862440zq
integers[x]	(-245z)+125z*X+65z*X^2	(-12300z)+9326z*X	2863877380z

3.2.3 PREM-based PRS

E_PRS(a, b): Euclidean polynomial remainder sequence.
I.e., a trace of the steps of Euclid's algorithm modified to use PREM.

```
E_PRS (a, b) ⇐ ⇑ [a] ||| (If =(b, 0(b)) then [b] else E_PRS(b, PREM(a, b))) ■
```

3.2.4 Subresultant PRS

The following algorithm is the Collins-Brown subresultant PRS algorithm, as presented in Yap [Yap85a].

S_PRS(p0, p1): Subresultant polynomial remainder sequence.

Input: polynomials $p_0, p_1 \in I[x]$ for some integral domain I.

Output: Subresultant PRS $(p_0, p_1, .. p_k)$ such that $p_{k+1}=0$.

Let $\delta_i = deg(p_i) - deg(p_{i+1})$.

Let $c_i = lead(p_i)$.

Let $(R_1, R_2 \cdots R_k)$ be a sequence of length k defined by

$R_1 = c_1^{\delta_0}$

$R_i = (c_i^{\delta_{i-1}} / R_{i-1}^{\delta_{i-1}-1}), i=2..k$

Let $(\beta_2, \beta_3 \cdots \beta_k)$ be a sequence of length $k-1$ defined by

$\beta_2 = (-1)^{(\delta_0+1)}$

$\beta_i = ((-1)^{(1+\delta_{i-2})}) c_{i-2}((R^{i-2})^{\delta_{i-2}}), i=3..k$

Then we wish to compute the sequence $(p_0, p_1, .. p_k)$ of length $k+1$ such that p_0 and p_1 are given polynomials, and

$$p_i = \frac{prem(p_{i-1}, p_{i-2})}{\beta_i}, i=2..k.$$

```
S_PRS (p_0, p_1) ⇐

local δ_0, β_2, p_2, x, P, R_1,   # Initial values
      δ_{i-2}, c_{i-2},     # Iterate values
      R_{i-2}, p_{i-2},
      p_{i-1}, β_i, p_i, l, z

δ_0 := δ_i(p_0, p_1)              # Sequence initial values
c0 := c_i(p_0)
β_2 := poly_of(exp(−(1(c0)), δ_0+1))
p_2 := P_i(p_0, p_1, β_2); z := 0(p_2)
if =(p_2, z) then ⇑ [p_0, p_1]
P := [p_0, p_1, p_2]
R_1 := exp(c_i(p_1), δ_0)
δ_{i-2} := δ_i(p_1, p_2)   # Loop iterate initialization, β_i
c_{i-2} := c_i(p_1)
R_{i-2} := R_1
p_{i-2} := p_1                    # for p_i
p_{i-1} := p_2
l := 3
repeat {      β_i := β_i (δ_{i-2}, c_{i-2}, R_{i-2})
              p_i := P_i (p_{i-2}, p_{i-1}, β_i)
              if =(p_i, z) then ⇑ P else P |||:= [p_i]
              p_{i-2} := p_{i-1}
              p_{i-1} := p_i
              c_{i-2} := c_i(p_{i-2})
              R_{i-2} := R_i(c_{i-2}, δ_{i-2}, R_{i-2})
              δ_{i-2} := δ_i (p_{i-2}, p_{i-1})      }
■
```

$\delta_i\ (p_i, p_{i+1}) \Leftarrow \Uparrow -_{deg}(deg_{poly}(p_i), deg_{poly}(p_{i+1}))$ ■

$c_i\ (p_i) \Leftarrow \Uparrow lead_{coef}(p_i)$ ■

```
R_i (c_i, δ_{i-1}, R_{i-1}) ⇐
 ⇑ ⊖(exp(c_i, δ_{i-1}),
          exp(R_{i-1}, −_deg(δ_{i-1}, 1)))
■
```

$\beta_i\ (\delta_{i-2},\ c_{i-2},\ R_{i-2}) \Leftarrow$
$\Uparrow$ poly_of($\otimes$($\otimes$(exp($-$(1(c_{i-2})),
$1+\delta_{i-2}$),
c_{i-2}),
exp(R_{i-2}, δ_{i-2})))
■

$P_i\ (p_{i-2},\ p_{i-1},\ \beta_i) \Leftarrow \Uparrow \oplus$(PREM($p_{i-2}$, p_{i-1}), β_i) ■

3.3 Power series and polynomial inversion and interpolation

Under this heading we provide the following facilities:

- Newton's method for construction of polynomials by interpolation.
- Fast Fourier Transform (FFT) and Interpolation (FFI).
- Newton's method for truncated power series inversion.

3.3.1 Newton's method for construction of polynomials by interpolation

NIA (rm_list): Newton's Interpolation Algorithm (CRA for F[x])

Input: [[ak, bk]] such that U(ak) = bk, U(x) $\in$ F[x] Output: U(x)

```
NIA (ab_list) ⇐
local ab_s, ab, a, b, Ux, Mx, c, σ
ab_s := copy(ab_list)
ab := pop(ab_s); a := ab[1]; b := ab[2]
Ux := poly_of(b)
Mx := 1(Ux)
every k := 1 to *ab_s do
{     Mx := ⊗(Mx, ⊖(poly([term(1(b), 1)]), poly_of(a)))
      ab := pop(ab_s); a := ab[1]; b := ab[2]
      c := ⊕(1(a), eval_poly(Mx, a))
      σ := ⊗(⊖(poly_of(b), poly_of(eval_poly(Ux, a))),
                poly_of(c))
      Ux := ⊕(Ux, ⊗(σ, Mx))                }
⇑ Ux
■
```

3.3.2 Fast Fourier Transform (FFT) and Interpolation (FFI)

FFT(N,a(x),omega,A): Fast Fourier transform

Input: integer N = 2^m, polynomial a(x) = sum(i=0, N-1, ai * a^i), primitive Nth root of unity omega Output: array A = (A0, ..., AN-1) where Ak = a(omega^k)

```
FFT (N, ax, omega) ⇐
 local A, n, bx, cx, ω^2, B, C, ω^k
 A := list(N, [])
 if N = 1                                        # basis
 then A[1]:= 0th_coef(ax)
 else { n := N/2                                 # binary split
        bx := poly_of_even_powered_terms(ax)
        cx := poly_of_odd_powered_terms(ax)
        ω^2 := exp(omega, 2)
        B := FFT(n, bx, ω^2)                     # recursive calls
        C := FFT(n, cx, ω^2)
        every k := 1 to n do
        {       ω^k := exp(omega, k-1)
                A[k] := ⊕(B[k], ⊗(ω^k, C[k]))
                A[k+n] := ⊖(B[k], ⊗(ω^k, C[k]))     }}
 ⇑A
■
```

Even powered terms.

```
poly_of_even_powered_terms(ax) ⇐
 local r
 r := []
 every t := lax.terms
 do if mod_integer(t.power, 2) = 0 then r |||:= [term(t.coef, t.power/2)]
 ⇑poly(r)
■
```

Odd powered terms.

```
poly_of_odd_powered_terms(ax) ⇐
 local r
 r := []
 every t := !ax.terms
 do if mod_integer(t.power,2)=1 then r |||:= [term(t.coef, (t.power - 1)/2)]
 if *r > 0 then ↑ poly(r) else ↑ 0(ax.terms[1])
■
```

FFI(N,B,omega): Fast Fourier interpolation

Input: integer N = 2^m, sample values B = (b0, ... bN-1), primitive Nth root of unity omega Output: a(x) = sum(i=0, N-1, ai x^i) where a(omega^k) = bk, k=0..N-1.

```
FFI (N, B, omega) ⇐
 local bx, C, ax
 bx := polynomialize(B)
 C := FFT(N, bx, ⊕(1(omega), omega))
 ax := polynomialize(⊕_vector scalar(C, modulo(N,13)))
 ↑ ax
■

polynomialize (B) ⇐
 local r, i
 r := []; i := 0
 every b := !B
 do { if not(=(b, 0(b))) then r |||:= [term(b, i)]
    i +:= 1 }
 ↑ poly(r)
■

⊕_vector scalar (V, x) ⇐
 local R, i
 R := list(*V); i := 1
 every v := !V do { R[i] := ⊕(V[i], x); i +:= 1 }
```

```
⇑ R
■
```

3.3.3 Newton's method for truncated power series inversion

NPSI (): Newton's Power Series Inversion Method

Input: a(t) mod t^(2^n) = sum(i=0,2^n-1,ai t^i), a0 ≠ 0. Output: x^(n)(t) = a(t)^-1 mod t^2^n

```
NPSI (at) ⇐
 local ax, xt, n
 ax := at.Poly
 xt := poly_of(0th_coef(ax))
 n := log2(*ax.terms)
 every k := 0 to n-1
 do xt := ⊕(⊕(xt, xt),
            −(⊗_poly(truncate(ax, 2^(k+1)), ⊗(xt, xt))))
 ⇑ tpower(truncate(xt, at.N), at.N)
■

log2 (x) ⇐
 local l
 l := 0
 while x > 1 do { x := x/2; l := l+1 }
 ⇑ l
■
```

3.4 A simple timer

A call to *settime*() initializes the timer.

A call to *showtime*() prints the elapsed time since *settime*() was invoked.

```
global timer

showtime () ⇐ pr{"[", &time - timer, " msecs]"} ■
settime () ⇐ timer := &time ■
```

Appendix. ICON Pretty Printer and Documentation Delaminator

The following documentation filter is inspired by Knuth's *Tex* (specifically the *LaTex* variant [Lampo83a,Knuth82a].

Blocks of comments are compiled as paragraphs. Paragraphs are demarcated by blank comment lines. Paragraphs are typeset with .lp. Code is set off with .nf, and .fi. We strip any leading white space from comment lines before further processing.

```
global command_line, last_line, cur_files, read_now, words

main(x) <=
 local fn
 words := table("")
 words["↑"] := "↑"
 words["■"] := "■"
 words["⊥"] := "⊥"
 command_line := x
 if *command_line > 0
 then { fn := command_line[1]
        load_user_keywords(fn||".keys")
        cur_files := [read_now := open(fn||".icn", "r")] }
 else cur_files := [read_now := &input]
 last_line := &null
 write(".so /usr2/ericson/euclid/ipp/std.me")
 process()
■

get_line() <=
 local x
 x := &null
 if st_line; ↑ x }
 else if x := read(read_now) then ↑ x
■
```

Reads lines until encountering end of file or ##end or ##end command.

```
process (command) ⇐
 local line
 while line := get_line() do if not process_line(line, command) then break
■

process_line (line, command) ⇐
 if line[1:3] == "##"
 then { if line[3:6] == "■"
        then { end_command(command, line[7:*line+1]); ⊥ }
        else do_command(line[3:*line+1]) }
 else if line[1] == "#" then write_line(line[2:*line+1])
 else pretty_print(line, command)
 ⇑
■
```

If command is non-null then ##end command should match command.

```
end_command (command, line) ⇐
 if f  then write(&errout, "ERROR: Mismatched END, wanted ",command,", got ", line)
■
```

For interpreting ## commands

```
do_command (line) ⇐
 local command, args
 x := (upto(~&lcase, line) | (*line + 1))
 command := line[1:x]
 args := line[x+1:*line+1]
 if not(y := proc("do_" || command, 2))
 then write(&errout, "ERROR: Unknown command: ", command)
 else y(args)
■
```

##list and ##end list.

```
do_list (args) ⇐
 local line
 write(".(l l F")
 while line := get_line()
 do  if line[1:3] == "##"
    then { if line[3:6] == "■"
           then { write(".)l"); end_command(command,line[7:*line+1]); ⊥ }
           else do_command(line[3:*line+1]) }
    else if line[1] == "#"
         then {  line := line[2:*line+1]
                 repeat if upto('    ', line[1])
                        then line := line[2:*line+1] else break
                 if *line > 0 then write("● ", line) else write() }
    else pretty_print(line, command)
 write(".)l")
■
```

##section <i> <title> and ##end section <i>.

Section nestings are relative to the file, from 1 on up. An ##include file's nestings are relative to the current level of the including file plus previous cumulative nesting. I.e., if cumulative nesting is 3, and nesting in the including file is 2, then 1 in the included file translates to 6 in the final output.

```
do_section (args) ⇐
 x := (upto(~('0123456789'), args) | (*args + 1))
 level := args[1:x]+0
 title := args[x+1:*args+1]
 write(".sh ", level, "
 write(".sp 2v0lp")
 process("section "||level)
■
```

##skip and ##end skip.

Deletes *everything* between skip and end skip.

```
do_skip (x) ⇐
 local line
 while line := get_line()
 do if line[1:3] == "##"
   then if line[3:6] == "■"
        then { end_command(command, line[7:*line+1]); break }
■
```

##include <file>.

Includes file. Home directory for includes within included file is home directory of file relative to current home directory. I.e., if you include foo/bar (.icn is assumed), and foo/bar includes dot/zot, then we look for foo/dot/zot. If -I switch is present, don't bother doing includes.

```
do_include (arg) ⇐
 local new_file
 cur_file := arg
 new_file := open(cur_file||".icn", "r")
 if /new_file then write("ERROR: couldn't open ", cur_file, ".icn")
 else { read_now := new_file
        push(cur_files, read_now)
        load_user_keywords(cur_file||".keys")
        process("include")              # until ■ of file
        close(pop(cur_files))
        read_now := cur_files[1]  }
■
```

##example and ##end example

Example paragraphs are left-justified and preceded by an appropriately numbered boldfaced "Example" keyword.

```
do_example (arg) <=
 writes("\fBExample.\fR  ")
 process("example")
■
```

##code and ##end code

Code is unjustified and Helveticized. Uncommented lines are processed as code. Commented lines bracketed by ##code are treated similarly; the purpose is to present code examples in the file that are not to be seen by the ICON compiler.

```
do_code (arg) <=
 local line
 write(".nf0fH ")
 while line := get_line()
 do if line[1:6] == "##■" then break
   else pretty_print_line(line[2:*line+1])
 write(".fl0fR ")
■
```

##equations and ##end equations

Typeset with TBL, one .EQ and .EN. per line, except that if the line is terminated by , continue equation on the next line.

```
do_equations (arg) <=
 write(".EQ")
 process("equations")
 write(".EN")
■
```

##quote and ##end quote

These are typeset with .(q and .)q.

```
do_quote (arg) <=
 write(".(q")
 process("quote")
 write(".)q")
■
```

##table and ##end table

Outputs .TS and .TE commands. Body is straight TBL.

```
do_table (args) <=
 write(".sp 4v0(c0TS")
 process("table")
 write(".TE0)c0)
■
```

For printing documentation lines. If the text following the # is white space, output a .lp

```
write_line (line) <=
 repeat if upto('      ', line[1]) then line := line[2:*line+1] else break
 if *line = 0 then write(".lp") else write(line)
■
```

For printing list lines.

```
plain_write_line (line) <=
```

```
repeat if upto('        ', line[1]) then line := line[2:*line+1] else break
write(line)
■
```

pretty_print(line): For printing code. Output a .nf. Pretty print lines until end-of-file or comment. Output a .fi. Write-line the comment if there was one.

```
pretty_print (l, command) ⇐
 local line
 write(".nf0fH ")
 pretty_print_line(l)
 while line := get_line()
 do if line[1:2] == "#"
   then {      write(".fl0fR ")
               write(".lp"); last_line := line; ⊥ }
   else pretty_print_line(line)
 write(".fl0fR ")
■
```

Pretty-print does special formatting in the following cases:

Procedure definitions Control structures Reserved words User keywords

If the -U<filename> option is present, then keywords are read into the words table, with troff equivalents.

```
pretty_print_line (line) ⇐
 local first, last, key, x, y
dure", line) ⇐ ⇐
        {      x := (upto((&lcase++&ucase++'_0123456789'), line) | *line+1)
               if x = *line+1 then { writes(line); break }
ocedure (line[*x+10:*line+1]||" ⇐") ⇐ ⇐
               y := (upto(~(&lcase++&ucase++'_0123456789'), line) | *line+1)
               key := (line[1:y] | "")
               first := (line[1:x] | "")
               line := (line[x:*line+1] | "")
```

```
else { while *line > 0 do
            if words[key] ~== "" then key := words[key]
            last := line[y:*line+1]
            writes(first, key)
            line := last }
     write() }
■

load_user_keywords(fname) ⇐
local w, a, x
if not(w := open(fname, 'r')) then ⊥
if 75while x := read(w)
        do { a := upto(':', x)
             words[x[1:a]] := x[a+1:*x+1]     }
close(w)
⇑
■
```

Procedure definitions. Instead of the obvious

```
     procedure F (a, b, c)
code
end
```

we use the logical-looking

```
F (a, b, c) ⇐ code ■


e (line) ⇐ ⇐ ⇐
            if z == "■" then pretty_print_line (line||y||" ■")
            else { pretty_print_line(line)
if y == "■" then pretty_print_line(line || "⊥■")
else { z := get_line()
local y
y := get_line()
```

```
pretty_print_line(y); pretty_print_line(z) } }
```

■

Control Structures: return, fail and every.

Instead of return x we use *uparrowx*, and for return we use *uparrow*. Instead of fail we use ⊥.

For every i := 1 to j do C every i := j to 1 by -1 do C and every x := !Y do C

we use every i in 1, 2..j do C every i in j, j-1 .. 1 do C end every x in Y do C

References

Balza84a.

Stepehn R. Balzac, James H. Davenport, Patrizia Gianni, Richard D. Jenks, Victor S. Miller, Scott C. Morrison, Michael Rothstein, Christine J. Sundaresan, Robert S. Sutor, and Barry M. Trager, *Scratchpad II: An experimental computer algebra system*, Mathematical Sciences Department, IBM Thomas J. Watson Research Center, Yorktown Heights, NY 10598, May, 1984.

Dewar81a.

Robert B.K. Dewar, Ed Schonberg, and Jacob T. Schwartz, *Higher level programming: Introduction to the use of the set-theoretic programming language SETL*, Courant Institute, N.Y.U., Summer, 1981.

Grisw83a.

Ralph E. Griswold, "An overview of the Icon programming language (revised, September, 1985)," TR 83-3a, Dept. of Computer Science, University of Arizona, May, 1983.

Grisw83b.

Ralph E. Griswold and Madge T. Griswold, *The Icon Programming Language*, Prentice-Hall, Inc., Englewood Cliffs, New Jersey, 1983.

Grisw85a.

Ralph E. Griswold and William H. Mitchell, *Version 5.10 of Icon*, TR 85-15, Dept. of Computer Science, University of Arizona, August, 1985.

Ingal78a.

D.H.H. Ingalls, "The SMALLTALK-76 programming system design and implementation," in *Fifth Annual ACM Symposium on Principles of Programming Languages*, pp. 9-16, 1978.

Knuth73a.

Knuth, *The art of computer programming*, 1973.

Knuth82a.

Knuth, Donald, "Web documentation system," *UNIX TeX Distribution Tape*, U. of Washington, 1982.

Kruch83a.

Philippe Kruchten and Edmond Schonberg, *The Ada/Ed system: a large-scale experiment in software prototyping using SETL*, Computer Science Department, Courant Institute, New York University, 251 Mercer St., NY, NY, 10012, 1983.

References

Lampo83a.

Lamport, Leslie, *The LaTex Document Preparation System*, 1983.

Lipso81a.

John D. Lipson, *Elements of Algebra and Algebraic Computing*, Benjamin/Cummings, 1981.

Loosa.

Loos, *Polynomial remainder sequences*, Computer Algebra (ed. Buchberger).

NYU 84a.

NYU Ada Project, *AdaSem: Static Semantics for Ada*, Ada Project, Courant Institute, New York University, 251 Mercer St., New York, NY, 10012, June, 1984.

Niven80a.

Ivan Niven and H.S. Zuckerman, *An introduction to the theory of numbers, 4th ed.*, John Wiley & Sons, 1980.

Yap85a.

Yap, Chee, *Polynomial remainder sequences and theory of subresultants*, Unpublished lecture notes, NYU Courant Institute, Fall, 1985.

Yap86a.

Chee Yap, *Root Isolation*, Unpublished lecture notes, N.Y.U., 1986.

Zippe86a.

Richard E. Zippel, *Algebraic Manipulation*, Unpublished lecture notes, M.I.T., 1986.